

6

プログラミング概論I (オンデマンド授業)



担当: 中山 功一 (7号館3階308室)

メールアドレス: knakayama@is.saga-u.ac.jp

506

オープニングテスト

前回, 習ったことを復習するために,
オープニングテストを行います

問題ごとに動画を止めて, 答えをメモしましょう.

最後に, 答えを動画で表示しますので,
自分が理解しているか, 確認しましょう.

オープニングテスト

使うのは①／②のどちら？

【問1】長いプログラムの特定の場所で確認する

【問2】プログラムを1行ずつ確認する場合

【問3】特定の条件でのみプログラムを止める

ブレークポイントは, プログラム中の
異なる場所に複数個の指定が【問4】

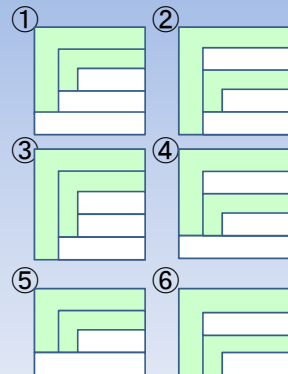
<選択肢> ① ブレークポイント,
② ステップ実行, ③ できる, ④ できない

508

オープニングテスト

```
void main(){  
    for(a=1; a < 10; a++){  
        cout<<a;  
        for(b=1; b < 10; b++){  
            cout<<b;  
        }  
    }  
    cout << endl;  
}
```

【問5】プログラムを
示すものを選びなさい



答え合わせ

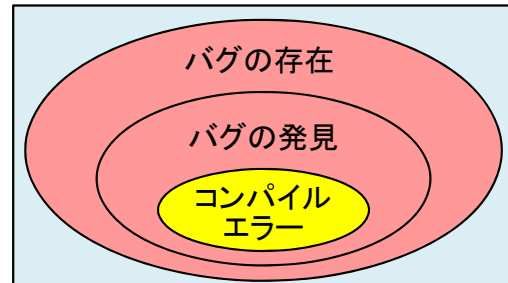
全ての答えをメモしましたか？

解答

- 【1】 ①
- 【2】 ②
- 【3】 ①
- 【4】 ③
- 【5】 ④

「バグ」に対する考え方

バグが存在する ⇔ バグが見つかる



512

「バグ」の存在証明

- 与えられたアルゴリズムが
有限時間で停止するかは判定できない
- 同様に、プログラムにバグがあるかどうかを
完璧に判定するプログラムは作れない
(アラン・チューリング)

バグが存在しないことは証明できない！
(バグが限りなく少ないプログラムを目指す)

513

バグの探し方の概念

バグ = 意図と異なる動きの原因, 不具合

- ・打ち間違い
- ・コーディングのミス
- ・アルゴリズム設計の誤り

など

バグのある場所(原因箇所)の探し方

大きな(長い)プログラムの場合,
バグのある場所の特定が難しい

<問題>

1万個の関数からなる
プログラムでバグXが発生した
原因特定として正しい方法は？

原因箇所の特定手順 その1

選択肢1: 部分的にコメントアウトして
バグが消える条件を探す

選択肢2: 部分的にコメントアウトしても
バグが消えない条件を探す

選択肢3: プログラムをトレースして
バグ発生時に**使用される関数**を探す

バグのある場所(原因箇所)の探し方

1万個の**関数**からなる

プログラムでバグXが発生した
原因特定として正しい方法は？



1万個の**遺伝子**からなる

iPS細胞で初期化(バグX)が発生した
原因特定として正しい方法は？

原因箇所を特定した人 サイエンスZERO:2012年NHK



原因箇所の特定手順 その1

<正解>

選択肢3:プログラムをトレースして
バグ発生時に使用される関数を探す

まずは原因となる可能性のある関数を

バグXが発生する状態で
デバッグしながら絞り込む

犯人はこの中にいる！

原因箇所の特定手順 その2

<問題>

バグX発生時に
使用された関数は24個でした

原因箇所の特定として
正しい次の手順は？

原因箇所の特定手順 その2

選択肢1:部分的にコメントアウトして
バグが消える条件を探す

選択肢2:部分的にコメントアウトしても
バグが消えない条件を探す

原因箇所を特定した人 サイエンスZERO:2012年NHK



原因箇所の特定手順 その2

<正解>

選択肢2:部分的にコメントアウトしても

バグが消えない条件を探す

(=バグが存在しない場所を順番に探す)

つまり, 原因となる場所を

アリバイ探し

バグが発生する状態で

コメントアウトしながら絞り込む

関数Aを削除してバグXが消えたら?

<正しい推定>

バグXは, 関数A以外に存在するかもしれない

(=関数AにバグXが存在する**必要条件**を満たす)

<間違った推定>

バグXは, 関数A以外には存在しない

(=関数AにバグXが存在する**十分条件**を満たす)

524

原因箇所の特定

✕ 関数A,B,Cをコメントアウトしたら
バグXが消えたので**関数A,B,C**にバグがある

○ 関数A,B,Cをコメントアウトしても
消えないので**関数A,B,C以外**にバグXがある

アリバイ成立

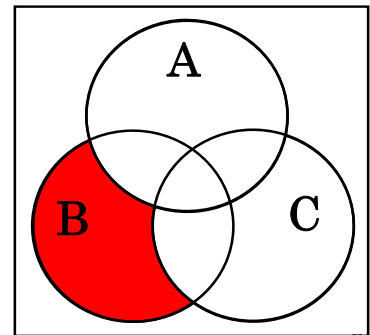
○ バグX発生時に**関数A,B,Cのみ**使用されたので
関数A,B,CのいずれかにバグXがある

犯人はこの中にいる

「バグX」の場所の推定方法

関数B単独で
バグX発生時

- Aを消す
⇒バグX存続
- Bを消す
⇒バグX消滅
- Cを消す
⇒バグX存続

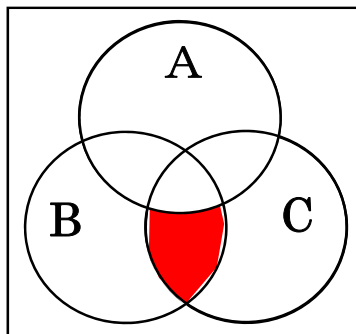


525

「バグX」の場所の推定方法

関数BとCにより
バグX発生

- Aを消す
⇒バグX存続
- Bを消す
⇒バグX消滅
- Cを消す
⇒バグX消滅



527

「バグX」の場所の推定方法

【誤り】 バグXが消える場所を探す
⇒ そこにバグXがあるとは限らない

【正しい】バグXが残る場所を探す
⇒ 残った中にバグXの原因がある

バグが残る最低条件を探す
(犯人はその中にいる!)

どんな名探偵も, 犯罪が発生しないと
犯人を見つけられない

528

2020年度の第1中間テストの解説

注: 2020年度の
第1中間テストはTechFULでした

第1中間テストの解説

- 問1
- 問2
- 問3
- 問4

530

第1中間テストの解説

- 問1
- 問2
- 問3
- 問4

531

第1中間テストの解説

- 問1
- 問2
- 問3
- 問4

532

第1中間テストの解説

- 問1
- 問2
- 問3
- 問4

533

534

行列の掛け算

行列の積

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11}b_{11}+a_{12}b_{21} & a_{11}b_{12}+a_{12}b_{22} & a_{11}b_{13}+a_{12}b_{23} & a_{11}b_{14}+a_{12}b_{24} \\ a_{21}b_{11}+a_{22}b_{21} & a_{21}b_{12}+a_{22}b_{22} & a_{21}b_{13}+a_{22}b_{23} & a_{21}b_{14}+a_{22}b_{24} \\ a_{31}b_{11}+a_{32}b_{21} & a_{31}b_{12}+a_{32}b_{22} & a_{31}b_{13}+a_{32}b_{23} & a_{31}b_{14}+a_{32}b_{24} \end{bmatrix}$$

536

i行j列の掛け算の計算

1行1列目: $a_{11}b_{11}+a_{12}b_{21}$

1行2列目: $a_{11}b_{12}+a_{12}b_{22}$

2行1列目: $a_{21}b_{11}+a_{22}b_{21}$

問題:i行j列目は?

$$a_{??}b_{??}+a_{??}b_{??}$$

537

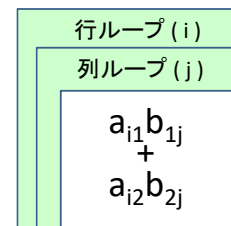
i行j列の掛け算の計算

i行j列目の計算は

$$a_{i1}b_{1j}+a_{i2}b_{2j}$$

(kをループ変数とすると

$a_{ik}b_{kj}$ と表現できるが
2個なので利用しない)



※データ入出力は省略

538

クロージング テスト

今回、習ったことを復習するために、
クロージングテストを行います

問題ごとに動画を止めて、答えをメモしましょう。

最後に、答えを動画で表示しますので、
自分が理解しているか、確認しましょう。

クロージング テスト

関数A、関数Bからなる
プログラムにバグXがあった
関数Aを消すとバグXが発生せず
関数Bを消すとバグXが発生した
このバグXの原因は、
【問1】関数Aにある？
【問2】関数Aにない？
【問3】関数Bにある？
【問4】関数Bにない？

① 正しい ② 正しいとは限らない

540

大きなプログラムをデバッグする場合、まず、【問5】という作業をする。その後、【問6】という作業によりバグの場所を特定する

- ①さまざまな関数をコメントアウトして、コメントアウトするとバグが発生しなくなる関数を探す
- ②さまざまな関数をコメントアウトして、バグが発生し続けるときの関数の最小の組合せを探す
- ③プログラムをデバッグして、バグ発生する時に使用されている関数を探す
- ④プログラムをデバッグして、バグが発生しない時に使用されていない関数を探す

541

答え合わせ

全ての答えをメモしましたか？

解答

- 【1】 ②
- 【2】 ②
- 【3】 ②
- 【4】 ①
- 【5】 ③
- 【6】 ②

お疲れさまでした

